AGENTFORCE HACKATHON - THEME 6

The Flowbreakers Automation Conflict Investigator

A customer-ready story for investigating Salesforce automation conflicts with traceable evidence and human-controlled change.

Jorge Felipe | November 2026

All organizations, people, records and automation names in this document are fictional. No client data or confidential business rule is included.

Executive summary

A single Case record type can accumulate record-triggered flows, Apex triggers, validation rules, email handlers and integration logic. When several components evaluate the same fields, a wrong owner, route or status may have many plausible causes. The administrator must reconstruct the business rule before changing production.

Flowbreakers proposes an Agentforce investigation experience that narrows the automation landscape, distinguishes possible dependencies from observed execution, explains the supported causal chain and prepares regression tests for human review.

From 34 possible automations to one evidence-backed explanation

Theme 6 positioning. This is a concept and presales-story exercise. The current two-flow laboratory is optional proof that ordered writes can be analyzed. Enterprise metadata discovery, dependency mapping and customer adapters belong to the proposed future state.

Business outcomes

Outcome	How to measure it
Faster onboarding	Time for a new team member to explain a selected automation path
Faster incident investigation	Median time to an accepted diagnosis
Safer change planning	Regression scenarios accepted by the business owner
More honest AI behavior	Correct requests for missing evidence and unsupported-claim rate

The business problem

Picture the first week of a Salesforce support project. Multiple teams have changed the org, the latest business rule is not documented in one place, and a Case reaches the wrong queue. The Case record type has 34 relevant automation components. Some share entry conditions. Several write the same fields. A background update may run under a different user context than the administrator who tests manually.

The difficult question is not "Which flow mentions OwnerId?" The difficult question is "Which automations could run for this record, which ones actually ran, what wrote the final value, and what else could break if we change the rule?"

Primary users

Persona	Goal	Current pain
Salesforce administrator	Explain an unexpected result and prepare a safe correction	Searches metadata, history and logs separately
Support developer	Locate the relevant code and automation boundary	Must infer business intent from implementation
Service owner	Confirm the intended rule and operational impact	Cannot approve a change from technical names alone
QA or release owner	Validate the regression scope before promotion	Receives fragmented evidence late in the process

Primary storyboard: competing Case routing

Step	What happens
1. Signal	A new complaint should reach Specialists but ends in Intake.
2. Context	The administrator supplies the Case, record type, expected route and authorized evidence sources.
3. Narrowing	The concept maps 34 components to 8 candidates, then identifies 3 writers of the investigated field.
4. Execution evidence	The trace shows SpecificRouting writing Specialists at E1 and BroadRouting writing Intake later at E2.
5. Explanation	The agent explains the ordered writes and identifies overlapping conditions as the review target.
6. Change brief	The agent drafts regression scenarios. A human confirms policy, implements the correction and controls release.

Important evidence boundary

Static metadata can identify candidates and dependencies. It cannot prove that a component ran in one transaction. The causal conclusion requires trace, field history, structured event evidence or another trusted runtime source.

The current laboratory proves a smaller mechanism: two active record-triggered flows write the same route field, the later write wins, and deterministic analysis links the explanation to ordered event IDs.

Proposed experience

Stage	User experience	Agent responsibility
Frame the incident	State the symptom, expected behavior and affected record	Confirm scope and request the minimum context
Map candidates	Review automation relevant to the object, record type, event and fields	Build a candidate set without claiming execution
Analyze evidence	Provide active versions, trace, history or integration logs	Order observed writes and link findings to evidence IDs
Plan the change	Review overlapping conditions and potential side effects	Draft a focused change brief and regression matrix
Control the release	Validate in sandbox and approve promotion	Remain read-only and preserve the audit trail

Future-state response pattern

Supported finding: “E1 selected Specialists. E2 wrote Intake later. Both events refer to the same Case transaction.”

Missing evidence: “Three components can write the field, but the active version and runtime trace are missing. I cannot identify the final writer yet.”

Human decision: “Review the overlap between the specific and broad rules. Test the qualifying complaint, general requests, manual reassignment and reopen behavior before release.”

Reusable fictional use cases

These cases are customer-neutral patterns for discovery workshops and presales demos. Each prospect supplies its own objects, rules and evidence sources.

UC-01 - Competing routing

Symptom	Evidence needed	Desired outcome
A before-save flow selects a specialist queue. An after-save flow matches a broader condition and overwrites the owner.	Ordered writes, entry conditions, active versions	Explain the overwrite and propose exclusion tests.

UC-02 - Reply reopens and reassigns

Symptom	Evidence needed	Desired outcome
A customer email reopens a closed Case but legacy logic restores an obsolete owner.	Email event, status history, owner history, handler logic	Preserve the current owner while reopening the Case.

UC-03 - Validation blocks automation

Symptom	Evidence needed	Desired outcome
A background update fails because a validation rule requires a field that users normally complete on screen.	Validation formula, changed fields, running user, failed transaction	Separate interactive requirements from approved automated transitions.

UC-04 - UAT succeeds, production fails

Symptom	Evidence needed	Desired outcome
The deployment reaches production, but a record type, picklist value or active validation differs from UAT.	Metadata diff, active values, deployment manifest	Identify configuration drift before blaming the flow.

UC-05 - Integration creates an incomplete Case

Symptom	Evidence needed	Desired outcome
An external payload creates a Case without a classification value expected by downstream automation.	Payload mapping, defaults, integration logs, downstream trace	Locate the missing contract field and the first failed dependency.

UC-06 - Manual transfer is undone

Symptom	Evidence needed	Desired outcome
An analyst transfers a Case, then a later update reruns routing and returns it to a queue.	Owner history, changed fields, routing conditions, transaction sequence	Protect deliberate manual ownership from unrelated updates.

Additional expansion cases

Case	Fictional problem	Presales value
Permission-context mismatch	An administrator test passes, but an integration or agent user cannot read a field or execute an action.	Explains why “works for admin” does not prove production access.
Scheduled job ownership risk	A critical job, dashboard or email service depends on a human user who will be deactivated.	Maps operational dependencies before access cleanup.
Status-value inconsistency	Automations compare different capitalization or legacy values for the same business state.	Finds hidden rule fragmentation and proposes canonical values.
Duplicate business rules	Flow, Apex and validation logic enforce overlapping versions of one policy.	Creates a business-rule inventory and identifies ownership gaps.
Missing regression coverage	A local correction passes its happy path but breaks reopen, reclassification or manual transfer.	Generates a scenario matrix tied to the proposed change.

Discovery questions

Which Case incidents consume the most investigation time? Which record types have the highest automation density? Where do manual transfers, email replies or integrations change the execution path? Which rules are known only by experienced team members? Which evidence can teams legally and reliably supply to an investigation agent?

Architecture and operating model

Layer	Proposed responsibility	Boundary
Metadata adapters	Collect authorized Flow, Apex, validation and configuration summaries	Candidate mapping only; metadata does not prove execution
Evidence normalizer	Convert traces, history and logs into stable event references	Reject malformed or incomplete evidence
Deterministic analysis	Order writes, compare expected and observed values, identify gaps	No deployment or data mutation
Agentforce experience	Explain findings, ask for missing evidence and draft tests	Treat retrieved content as untrusted data
Human review	Confirm the business rule, implement, test and approve release	Retains accountability for every change

Current optional prototype

A published Agentforce Employee agent loads synthetic evidence, reads authorized laboratory snapshots and calls deterministic Apex analysis. Two flows reproduce a routing overwrite. The prototype validates the conversation pattern and safety boundary. It does not yet inventory an enterprise org or diagnose every scenario in this concept.

A compelling presales story can show the full future-state journey while labeling the prototype as optional technical evidence.

Success measures and value assumptions

The presenter has experienced first-pass investigations that take two or three hours in complex projects. This observation is useful context, but a pilot must measure it rather than present it as a universal benchmark.

Measure	Baseline to capture	Pilot signal
Time to accepted diagnosis	Manual review time for matched incidents	Median reduction with reviewer acceptance
Evidence quality	Claims without traceable support	Every causal claim links to an evidence reference
Abstention quality	Incorrect conclusions on incomplete cases	Agent requests the missing version, trace or context
Regression usefulness	Tests rewritten or rejected by reviewers	Business and QA accept the proposed scenario scope
Safety	Unauthorized reads, writes or deployment attempts	No unauthorized exposure or change capability

Illustrative planning case

If a complex manual first pass takes 120 minutes and the assisted process produces a reviewable brief in 30 minutes, the planning hypothesis is 90 minutes of capacity per investigation. The pilot must include evidence collection and human review time. Generated text alone does not count as resolution.

Pilot and next steps

Pilot element	Recommendation
Evaluation set	20 fictional incidents across five patterns. Include sufficient, ambiguous and deliberately incomplete evidence.
Reviewers	Salesforce administrator, support developer, business owner and QA or release representative.
Comparison	Use matched incidents and compare manual review with assisted review.
Acceptance	Faster accepted diagnosis, correct abstention, useful regression scope and no unauthorized action.
Reuse	Keep the evidence contract stable. Configure object adapters, field mappings and business vocabulary per prospect.

Recommended demo flow

Open with the 34-automation Case problem. Show the investigation funnel. Walk through the two-writer evidence conversation. Present two additional patterns to prove reuse. Close with the human-controlled change brief and measurable pilot.

The customer-ready promise

Flowbreakers helps teams understand an unfamiliar Salesforce org faster. It turns scattered technical artifacts into a traceable explanation, a list of evidence gaps and a regression plan that business and technical reviewers can discuss together.

Risks, assumptions and references

Area	Boundary or mitigation
Causality	Do not infer execution from static references alone. Require runtime evidence for causal claims.
Data access	Use authorized sources, least-privilege identities and read-only investigation actions.
Prompt injection	Treat logs, metadata descriptions and retrieved documents as untrusted content.
Business intent	A business owner confirms the expected rule before correction.
Environment drift	Capture active versions, record types, picklists and validations for the target environment.
Prototype scope	Current native analysis covers supplied ordered routing evidence, not automatic enterprise discovery.

Primary public references

Salesforce Architects, “Record-Triggered Automation.”
https://architect.salesforce.com/docs/architect/decision-guides/guide/record-triggered

Salesforce Developers, “Agent Script Decoded: How to Debug Agent Script.”
https://developer.salesforce.com/blogs/2026/03/agent-script-decoded-how-to-debug-agent-script

Salesforce Developers, “Design Security-Hardened Prompts.”
https://developer.salesforce.com/docs/platform/isvforce/guide/secure-agentforce-prompt-injection-harden.html

Disclosure

All business scenarios in this book are fictional and anonymized. They combine common Salesforce delivery patterns for presales storytelling. No customer-specific names, records, credentials, endpoints or restricted business rules appear in this document.